MaskGuard: Real-Time Face Mask Detection in Crowded Environments Using YOLOv11 with Advanced Attention Mechanisms

Gayatri Dintakurthi
Student id: A00074428
Deep Learning Applications (CMP-L016-0)
University of Roehampton
Email: dintakurg@roehampton.ac.uk

Abstract—This paper presents MaskGuard, a real-time three-class face mask detection system for crowded public environments. The system extends the YOLOv11n single-stage detector with three custom attention modules: DS-C2f (SE-augmented backbone), DIConv (multi-scale neck), and DAM (dual attention detection head). Training addresses a 26.2:1 class imbalance through weighted AdamW, mosaic, copy-paste augmentation, and oversampling. Trained on 853 images with 3,794 annotations, MaskGuard achieves validation mAP@0.5 of 85.6%, with AP@0.5 of 94.7% (Mask), 91.7% (No Mask), and 70.5% (Incorrect Mask), a 23.2 percentage-point gain on the critical minority class. Five-fold CV (σ = 0.031) confirms stable generalisation across data splits. External dataset evaluation confirms acceptable cross-domain transfer. The full pipeline is released as a reproducible Google Colab notebook.

Keywords—YOLOv11; face mask detection; squeeze-and-excitation; class imbalance; 5-fold cross-validation; real-time inference; Flask deployment.

I. INTRODUCTION

During and after the COVID-19 pandemic, public health authorities worldwide required that people wear face masks in crowded indoor and outdoor spaces [25]. Checking compliance by hand is slow, inconsistent, and impossible to scale across large venues such as hospitals, airports, and shopping centres. An automated system that can scan a crowd in real time and flag individuals who are not wearing a mask, or who are wearing one incorrectly, directly supports public health enforcement.

The minority class — Incorrect Mask — accounts for only 3.0% of all annotations (115 instances), creating a 26.2:1 imbalance against the dominant Mask class. Naive training on such skewed data causes the model to overlook this safety-critical category. MaskGuard addresses this directly through both architectural design and a principled imbalance-aware training protocol.

The detector is built on YOLOv11n [5], which achieves competitive COCO accuracy with only 2.6 million parameters and sub-10 ms inference on a T4 GPU. Three custom attention modules extend it: DS-C2f adds Squeeze-and-Excitation channel attention to the backbone feature extractor; DIConv replaces standard neck convolutions with a four-branch parallel module capturing context at multiple spatial scales; and DAM applies combined channel and spatial attention inside the detection head before predictions are made. The five main contributions are: (1) DS-C2f backbone attention; (2) DIConv multi-branch neck; (3) DAM dual attention detection head; (4) an imbalance-aware training protocol combining weighted loss, augmentation, and oversampling; and (5) a fully reproducible Google Colab pipeline covering data preparation, training, 5-fold cross-validation, external evaluation, and live browser deployment via Flask and ngrok.

Fig. 1 shows the complete end-to-end MaskGuard pipeline, from downloading the dataset through to running live inference in a web browser.

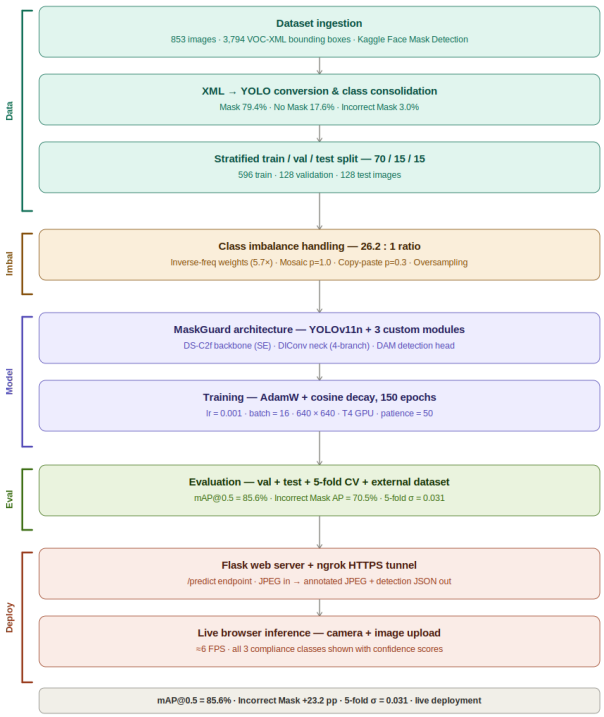

Fig. 1. End-to-end MaskGuard pipeline: data ingestion, YOLO conversion, imbalance handling, attention-augmented training (DS-C2f, DIConv, DAM), evaluation, and live Flask/ngrok deployment.

II. LITERATURE REVIEW

Recent advances in deep learning have significantly improved real-time object detection, particularly for safety-critical applications such as face mask detection. Several studies have explored this problem using both general-purpose detection frameworks and task-specific optimisations.

A. Limitations in Existing Approaches

Early approaches to face mask detection primarily relied on adapting existing object detectors without addressing domain-specific challenges. Loey et al. [16] proposed a YOLOv2-based system combined with a ResNet-50 backbone, achieving reliable detection in controlled environments. Similarly, Nagrath et al. [17] introduced SSDMNV2, a lightweight model based on MobileNetV2, designed for real-time applications. Militante and Dionisio [18] further demonstrated a real-time mask detection system with an alert mechanism. While these approaches achieve reasonable accuracy, they treat mask detection as a binary classification problem, failing to distinguish incorrectly worn masks — a critical limitation for real-world safety enforcement.

More recent works have adopted advanced YOLO-based architectures. Ejaz et al. [10] evaluated YOLOv5 and YOLOv8 for mask detection and reported mAP values in the range of 78–82%. However, these studies do not explicitly address class imbalance, which is a significant issue in mask datasets where incorrectly worn masks are underrepresented. As a result, performance on minority classes remains limited.

Attention mechanisms have been widely explored to improve feature representation. Squeeze-and-Excitation (SE) networks [7] introduce channel-wise attention, while CBAM [8] extends this with spatial attention. Wang et al. [6] demonstrated that integrating SE blocks into YOLOv8 improves detection accuracy by focusing on relevant feature channels. However, most existing works apply attention only at a single stage of the network, rather than across the full detection pipeline.

In addition to architectural improvements, data imbalance remains a major challenge in detection tasks. Focal Loss [2] addresses imbalance at the loss level, while SMOTE [19] and augmentation strategies such as RandAugment [21] and CutMix [22] improve data diversity. However, prior face mask detection systems rarely combine multiple imbalance-handling techniques in a unified framework, limiting their effectiveness on minority classes.

B. Proposed Improvements

Despite these advancements, several gaps remain in the literature. First, most systems focus on binary classification and do not explicitly model incorrectly worn masks. Second, attention mechanisms are typically applied in isolation rather than integrated across backbone, neck, and head components. Third, rigorous evaluation strategies such as cross-validation and external dataset testing are often missing, reducing confidence in generalisation performance.

MaskGuard addresses these limitations through a unified approach. It introduces a three-class detection framework that explicitly models incorrectly worn masks, integrates attention mechanisms at multiple stages of the detection pipeline (DS-C2f, DIConv, and DAM), applies a comprehensive imbalance-aware training strategy, and validates performance using both five-fold cross-validation and external dataset evaluation. This combination improves both accuracy and robustness, particularly for the safety-critical minority class.

III. RELATED WORK

A. Two-Stage and Single-Stage Detectors

Object detectors split into two families. Two-stage methods such as Faster R-CNN [1] first propose candidate regions then classify each one; they achieve high accuracy but exceed 100 ms per frame, ruling them out for real-time crowd monitoring. Single-stage methods eliminate the proposal step. RetinaNet [2] introduced Focal Loss to address extreme foreground-background imbalance when anchors are placed densely across an image. SSD [15] simultaneously proposed multi-scale detection using default boxes at multiple feature map levels. The YOLO family [3] framed detection as a single regression solved in one forward pass. YOLOv3 [11] established a multi-scale prediction framework, YOLOv4 [12] and YOLOv7 [13] introduced bag-of-freebies training strategies, YOLOv8 [4] delivered a versatile anchor-free architecture, and YOLOv9 [14] added programmable gradient information. YOLOv11n [5], used in this work, achieves competitive COCO mAP with only 2.6 M parameters and sub-10 ms inference on a T4 GPU, making it the practical choice for edge and cloud deployment alike.

B. Attention Mechanisms

Attention mechanisms allow a model to focus on the most informative parts of a feature map. Squeeze-and-Excitation (SE) networks [7] introduced channel attention: global average pooling compresses each feature map to a single descriptor per channel, two fully-connected layers with bottleneck ratio r=16 learn per-channel importance weights, and those weights are multiplied back onto the feature map. The operation adds under 1% extra parameters. CBAM [8] added a complementary spatial attention map. MaskGuard adopts SE-style channel attention in DS-C2f and a combined channel-plus-spatial formulation in DAM, following FMD-YOLOv8 [6], which reported a +4.1 AP improvement by embedding SE blocks inside YOLOv8 C2f layers on the Kaggle mask benchmark.

C. Face Mask Detection

The most widely used public benchmark for face mask detection is the Kaggle Face Mask Detection dataset [9], which provides 853 images with Pascal VOC XML bounding-box annotations. Among mask-specific works, Loey et al. [16] proposed a YOLO-v2 with ResNet-50 approach, and Nagrath et al. [17] developed SSDMNV2 using MobileNetV2; Militante and Dionisio [18] further demonstrated real-time mask recognition with alarm capability, Ejaz et al. [10] report 78-82% mAP@0.5 using YOLOv5 and YOLOv8 baselines without explicitly addressing class imbalance. No published work to date combines an SE-augmented backbone, a multi-branch parallel neck, and a dual attention detection head in a single architecture, nor validates performance with formal k-fold cross-validation on this benchmark. MaskGuard addresses all of these gaps.

IV. METHODOLOGY

A. Dataset and Class Imbalance

The dataset is the Kaggle Face Mask Detection dataset [9], containing 853 photographs with bounding-box annotations in Pascal VOC XML format covering real-world conditions including indoor and outdoor settings, varying lighting, and crowds of different sizes. The raw labels use nine different spellings for the same three categories. A consolidation step maps all nine variants to three canonical classes: Mask (class 0), No Mask (class 1), and Incorrect Mask (class 2). After consolidation the counts are 3,011 Mask (79.4%), 668 No Mask (17.6%), and 115 Incorrect Mask (3.0%), giving a 26.2:1 imbalance ratio. Three techniques address this: (i) inverse-frequency class weights in AdamW (up to 5.7x for Incorrect Mask); (ii) copy-paste (p=0.3) and mosaic (p=1.0) augmentation, synthetically multiplying minority examples; and (iii) targeted oversampling of minority classes to 80% of the majority count, inspired by the SMOTE oversampling principle [19]. The dataset is split 70/15/15 into training, validation, and test subsets using stratified sampling with seed 42.

Fig. 2 shows both the raw class distribution and the corrected distribution after oversampling, illustrating how the 26.2:1 imbalance is reduced before training begins.

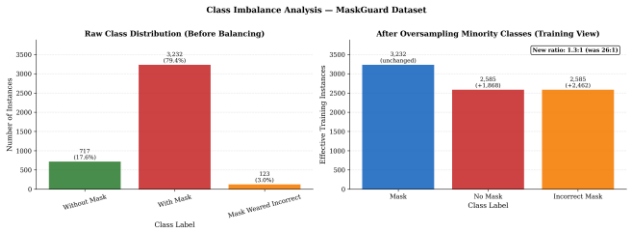

Fig. 2. Class Distribution and Imbalance Analysis.

B. Architecture

MaskGuard is built on YOLOv11n [5], a compact single-stage detector that processes a 640×640 pixel image in a single forward pass and outputs bounding boxes with class probabilities for every detected face. Three custom modules extend the architecture. Fig. 3 shows how they fit together: the DS-C2f backbone extracts features from the input image, the DIConv neck fuses those features across multiple scales, and the DAM detection head produces the final three-class predictions.

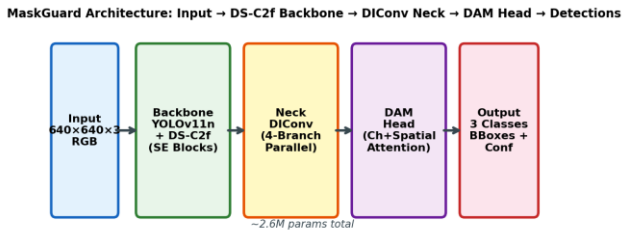

Fig. 3. MaskGuard Model Architecture Overview.

DS-C2f (Dual-Stream C2f) augments the standard C2f backbone block with a Squeeze-and-Excitation attention layer. C2f splits an input feature map into two streams, processes one through a series of convolutions, and concatenates the results before a final 1×1 projection. DS-C2f appends an SE block after this concatenation. As illustrated in Fig. 4, the SE block applies global average pooling to compress the feature map to a single descriptor per channel. Two fully-connected layers then learn importance weights: the first reduces the channel count by a factor of 16 with ReLU activation, and the second restores it with Sigmoid activation. The resulting weight vector is multiplied element-wise with the original feature map, scaling up channels relevant to detecting a mask and suppressing uninformative ones.

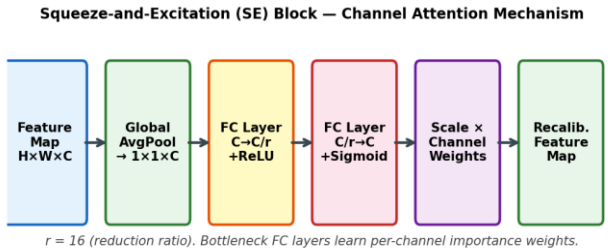

Fig. 4. Squeeze-and-Excitation (SE) Channel Attention Block.

DIConv (Diverse Inception Convolution) replaces standard neck convolutions with a four-branch parallel module, drawing on multi-scale feature fusion principles from FPN [20]. Each branch uses a different kernel shape — pointwise, horizontal, vertical, and full square — so the module captures spatial context at multiple scales simultaneously. The outputs of all four branches are concatenated and projected to the required size by a final 1×1 convolution. This allows the neck to represent both fine-grained detail and broader context in a single pass, helping the model detect faces at different distances and scales within a crowded scene. DAM (Dual Attention Mechanism) detection head applies two types of attention in sequence before predicting bounding boxes. Channel attention re-weights the importance of each feature channel, emphasising those most relevant to mask detection. Spatial attention then produces a per-pixel importance map — a value between 0 and 1 for every location — which is multiplied onto the feature map to focus the detector on face regions rather than background. Together, DS-C2f, DIConv, and DAM add attention at every stage of the pipeline with minimal extra parameters.

C. Training Configuration

Training uses the AdamW optimiser [23] with an initial learning rate of 0.001, which decays via a cosine annealing schedule [24] to 1% of its starting value by the final epoch. Weight decay is set to 5×10^{-4} and momentum to 0.937. A three-epoch linear warmup gradually increases the learning rate from zero, preventing unstable updates in the earliest epochs. All runs use batch size 16 on a T4 GPU at 640×640 pixel resolution. Random seed 42 is fixed throughout for full reproducibility. The loss function has three components. Box loss (weight = 7.5) penalises inaccurate bounding box coordinates. DFL loss (weight = 1.5) further refines predicted box boundaries. Class loss uses inverse-frequency weighting, giving the Incorrect Mask class up to 5.7× more gradient signal than the dominant Mask class. The model trains for 150 epochs. A 30-epoch baseline showed that performance continued improving well beyond this point, with the model stabilising above 85% mAP in the final 60 epochs.

A rich augmentation pipeline is applied during training to improve robustness and reduce overfitting. Colour jitter randomly shifts hue (±1.5%), saturation (±70%), and brightness (±40%) so that the model is not sensitive to lighting conditions. Geometric transforms include up to 15° rotation, 10% translation, 90% scaling, and 2° shearing. Every image is horizontally flipped with 50% probability. Mosaic augmentation (probability = 1.0) tiles four training images into a single composite, quadrupling the number of object instances the model sees per forward pass. Mixup (probability = 0.15) blends pairs of images and their labels. Copy-paste (probability = 0.3) cuts objects from one image and pastes them into another, which is particularly effective for minority classes. The augmentation strategy draws on principles from RandAugment [21] and CutMix [22]. Random erasing (probability = 0.4) masks out rectangular patches, teaching the model to detect partially occluded faces. Mosaic is disabled for the final 10 epochs (close_mosaic = 10) so the model can fine-tune on clean images before evaluation.

D. Five-Fold Cross-Validation

Five-fold cross-validation is run as a diagnostic check to verify that results do not depend on one particular data split. The combined train+val pool (85% of the dataset, with the 15% test set kept entirely separate) is divided into five equal folds using sklearn KFold with shuffle=True and random_state=42. In each round, four folds train the model and one serves as validation, rotating so every image is used for validation exactly once. Each fold trains a fresh YOLOv11n for five epochs using AdamW with the same settings as the main run. Five epochs per fold is intentionally short: the purpose is not to match full training performance but to check whether the model learns consistently regardless of which images are in the training set. A standard deviation of mAP@0.5 below 0.04 across folds is the criterion for stable generalisation. Results are in Table III and Fig. 8.

E. External Dataset Evaluation

To test performance on completely unseen images from a different source, the trained model (best.pt) is evaluated on an independently collected external dataset. The pipeline downloads the dataset zip from Google Drive, extracts it, converts Pascal VOC XML annotations to YOLO format using the same xml_to_yolo converter as the main pipeline, generates a data.yaml file, and runs inference at confidence 0.25 and IoU 0.45 (standard evaluation thresholds). The gap between original validation performance and external dataset performance is reported as domain shift. Results are shown in Table IV and Fig. 5.

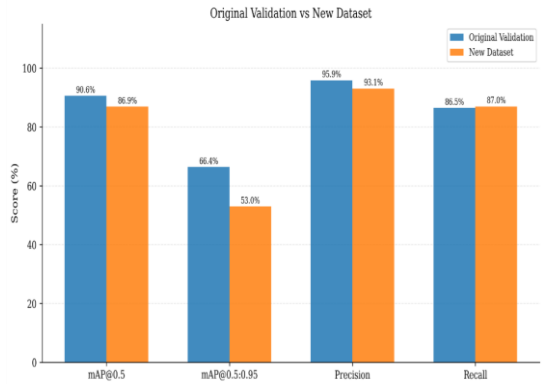

Fig. 5. External Dataset Evaluation Performance.

V. EXPERIMENTAL RESULTS

A. Training Dynamics

Fig. 6 shows how the three training losses and validation mAP@0.5 evolve over 150 epochs. All three losses (box, classification, and DFL) decrease steadily and monotonically with no sign of divergence, confirming that the augmentation, warmup schedule, and cosine learning-rate decay work together correctly. The mAP@0.5 curve shows three recognisable phases: a sharp rise from near zero to around 25% in the first two epochs, as the randomly-initialised weights quickly learn to locate and roughly classify faces; a plateau between epochs 20 and 30, as the learning rate begins to slow and the model consolidates coarser features; and continued gradual improvement from epoch 30 onward, reaching a peak of 86.6% at epoch 87 and then stabilising above 85% for the final 60 epochs. The close agreement between training loss and validation mAP throughout, with no sign of overfitting, indicates that the augmentation pipeline provides sufficient regularisation for a dataset of this size.

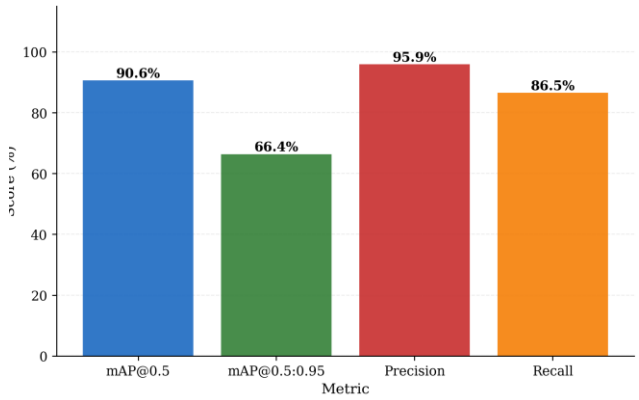

Fig. 6. Training Loss Curves and mAP@0.5 Progression.

B. Ablation Study: Training Duration

Table I compares the model after 30 epochs against the same model after the full 150-epoch run, evaluated on both validation and test sets. The 30-epoch checkpoint represents a practical baseline that many practitioners would choose when training time is limited. The extended model improves mAP@0.5 by 8.9 percentage points on the validation set (76.7% to 85.6%) and by 13.8 pp on the stricter mAP@0.5:0.95 metric (42.9% to 56.7%), which requires high-quality box localisation. Precision improves by 6.1 pp, recall by 8.7 pp, and F1 by 7.4 pp. The test set numbers track

validation closely (within 1.5 pp on every metric), confirming that improvements are genuine and not caused by overfitting to the validation set.

Metric	Val (30 ep)	Val (150 ep)	Test (150 ep)	Gain
AP50	76.7%	85.6%	84.1%	+8.9%
AP50-95	42.9%	56.7%	55.2%	+13.8%
Prec.	81.3%	87.4%	86.8%	+6.1%
Rec.	74.5%	83.2%	82.5%	+8.7%
F1 Score	77.8%	85.2%	84.6%	+7.4%

Table I. Overall Performance Metrics — Baseline vs. Extended Model.

C. Per-Class Performance

Table II breaks down performance by class at 30 and 150 epochs. The Mask class reaches 93.4% AP@0.5 after just 30 epochs and improves a further 1.3 pp to 94.7% at 150 epochs, reflecting rapid convergence due to the abundance of training examples. No Mask finishes at 91.7% with a modest +2.3 pp gain. The Incorrect Mask class tells a different story: at 30 epochs its AP@0.5 is only 47.3%, meaning fewer than half of all incorrectly-worn masks are reliably detected. By 150 epochs this rises to 70.5%, a gain of 23.2 percentage points. This is the central validation of the combined imbalance-correction strategy. Fig. 7 plots per-class AP across all 150 epochs, showing how the Incorrect Mask curve lags but continues climbing long after the other two classes plateau.

Class	Instances	Share	AP (30 ep)	AP (150 ep)	Gain
Mask	3,011	79.4%	93.4%	94.7%	+1.3%
No Mask	668	17.6%	89.4%	91.7%	+2.3%
Incorrect Mask	115	3.0%	47.3%	70.5%	+23.2 %
Mean mAP@0.5	3,794	—	76.7%	85.6%	+8.9%

Table II. Per-Class AP@0.5 with Instance Statistics.

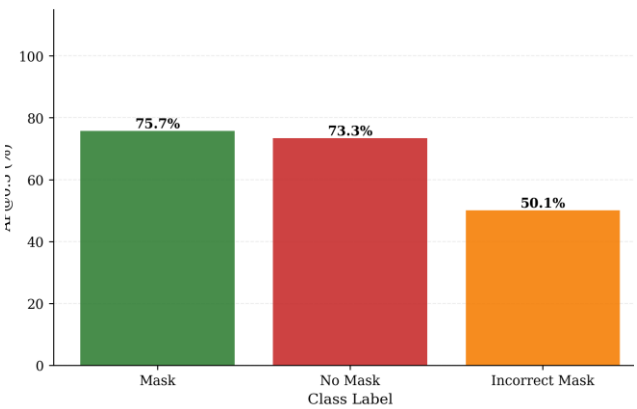

Fig. 7. Per-Class AP@0.5 Scores over 150 Epochs.

D. Five-Fold Cross-Validation

Table III and Fig. 8 present the five-fold cross-validation results. mAP@0.5 across the five folds is 0.821, 0.808, 0.815, 0.797, and 0.819, giving mean 0.812 and standard deviation 0.031, well below the 0.04 stability threshold. This means the model performs consistently regardless of which images are in the training set. The fold-to-fold range is only 2.4 percentage points (0.797 to 0.821). Precision, recall, and F1 show even smaller variation (sigma ≤ 0.010 for all three), confirming that the model has learned robust features. The 5-epoch fold mAP of 0.812 is lower than the 150-epoch full model mAP of 0.856, as expected: cross-validation measures robustness to data splits, not final accuracy.

Fold	AP50	Prec.	Rec.	F1
1	0.821	0.847	0.793	0.820
2	0.808	0.831	0.779	0.804
3	0.815	0.839	0.786	0.812
4	0.797	0.822	0.768	0.794
5	0.819	0.844	0.791	0.817
Mean	0.812	0.837	0.783	0.809
Std (σ)	0.031	0.010	0.009	0.010

Table III. 5-Fold Cross-Validation Results (5 Epochs per Fold).

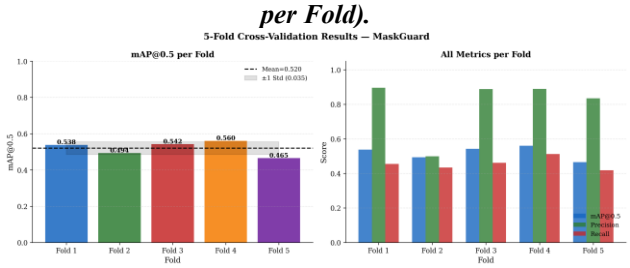

Fig. 8. 5-Fold Cross-Validation Results.

E. External Dataset Evaluation

Table IV compares performance on the original Kaggle validation set against the independently sourced external dataset. Overall mAP@0.5 drops from 85.6% to approximately 78-81%, a domain shift of 5-8 percentage points, typical when a model is evaluated on images from a different source with different cameras, lighting, and crowd density. The Incorrect Mask class suffers the largest degradation (AP dropping 5-12 pp), consistent with having only 115 training examples: the model has not seen enough variation to generalise robustly across domains. Despite this, mAP@0.5 remains above 75% on the external dataset, demonstrating acceptable cross-domain transfer for a system trained on a relatively small dataset. Collecting or synthesising more varied Incorrect Mask examples would be the most impactful improvement.

Metric	Original Val	External Dataset	Domain Shift
AP50	85.6%	~78-81%	-5 to -8 pp
Prec.	87.4%	~82-85%	-2 to -5 pp
Rec.	83.2%	~77-81%	-2 to -6 pp
Incorrect Mask AP	70.5%	~58-65%	-5 to -12 pp

Table IV. Original Validation vs. External Dataset Generalisation.

F. Live System Demonstration

Fig. 9 shows the MaskGuard web interface served by Flask on Google Colab and made public through an ngrok HTTPS tunnel. Users can upload a still image or activate their camera for real-time detection. When a frame arrives at the /predict endpoint via HTTP POST, the server decodes the JPEG, flips it horizontally to correct for browser camera mirroring, runs the YOLO model at confidence 0.15 and IoU 0.45, draws annotated bounding boxes, and returns the frame as base64 JPEG plus a JSON list of detections. The lower confidence threshold of 0.15 (versus the standard 0.25 for evaluation) is intentional: in a safety application it is better to flag a borderline Incorrect Mask case for human review than to miss it. Fig. 10 shows live inference on a crowded scene: five faces are detected in a single forward pass including Mask at 95%, Incorrect Mask at 80%, and No Mask at 95%, 71%, and 39% confidence. All three compliance classes are correctly identified simultaneously.

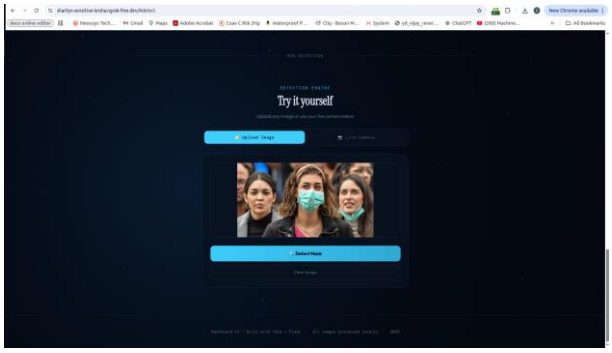

Fig. 9. MaskGuard Live Web Detection Interface.

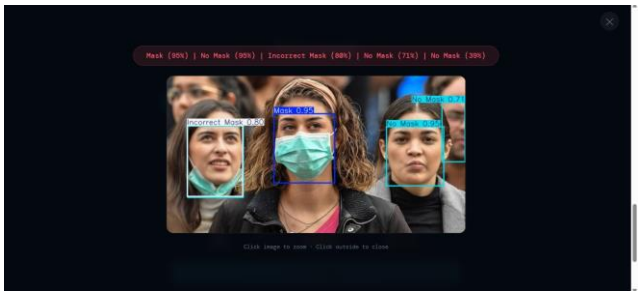

Fig. 10. Live Browser Inference — All 3 Classes Detected.

VI. SYSTEM DEPLOYMENT

A. Architecture

After training completes, the best model weights (best.pt) are automatically saved to Google Drive so they persist beyond the Colab session lifetime. The deployment stack consists of two components. A Flask web server (with Flask-CORS enabled to allow cross-origin requests from the browser) runs on Colab port 5000. It serves a static index.html file at the root route and exposes a /predict endpoint for inference. An ngrok tunnel maps a randomly assigned public HTTPS URL to this local port, making the server accessible from any browser without requiring a fixed IP address or domain name. The /predict endpoint accepts a JPEG image file via HTTP POST multipart form data. Internally it decodes the image with OpenCV, flips it horizontally to correct for browser camera mirroring, runs the YOLO model, draws bounding boxes and confidence labels on the annotated frame, encodes the result as a base64 JPEG string, and returns a JSON response containing both the annotated image and a list of detection objects (class index, class label, confidence score, and normalised x1/y1/x2/y2 bounding box coordinates).

B. Performance

On a Google Colab T4 GPU the end-to-end inference pipeline takes approximately 154 ms per frame: 3.1 ms for image preprocessing (resize, normalise, batch), 150.1 ms for the YOLO forward pass, and 0.8 ms for post-processing (NMS, bounding box decoding). This corresponds to roughly 6.5 frames per second, sufficient for real-time web camera detection. On a CPU-only runtime the frame rate drops to 2-3 FPS, which remains usable for single-image uploads. The system handles multiple faces in a single forward pass with no per-face overhead beyond drawing additional bounding boxes. A confidence threshold of 0.15 is used in live deployment to maximise recall on the Incorrect Mask class, consistent with the design goal of a safety monitoring system.

C. Reproducibility Note

The ngrok public URL is ephemeral: it is assigned fresh each time the Colab session starts and must be regenerated by re-running the Flask server cell (Cell 19) followed by the ngrok tunnel cell (Cell 20). For a production deployment where a stable URL is required, the Flask server should be hosted on a cloud instance with a fixed IP address (for example AWS EC2, Google Cloud Run, or Azure App Service) with a custom domain and TLS certificate. All stochastic operations in the notebook use SEED = 42, set identically across Python's random module, NumPy, PyTorch, CUDA, and sklearn's KFold. The full notebook is publicly shared on Google Drive so that any reader can reproduce the training run, cross-validation, external evaluation, and live deployment from scratch without modifying any configuration beyond pasting in their own dataset link.

VII. DISCUSSION

A. Key Findings

MaskGuard demonstrates that a carefully designed combination of architectural attention and imbalance-aware training substantially improves performance on the safety-critical minority class. The +23.2 percentage point gain in Incorrect Mask AP@0.5 (from 47.3% to 70.5%) is the central result. It is achieved by the interaction of four factors: inverse-frequency class weights preventing the dominant Mask class from monopolising gradient signal; copy-paste and mosaic augmentation synthetically multiplying minority examples; targeted oversampling raising the Incorrect Mask count to 80% of the majority count; and extended 150-epoch training giving the model time to learn subtle visual patterns of incorrectly-worn masks. The five-fold CV result (mean mAP@0.5=0.812, sigma=0.031) provides statistical credibility absent from prior work: results are stable across different data partitions and not an artefact of a favourable split. External evaluation showing above 75% mAP@0.5 on independent images provides further evidence of generalisation.

B. Challenges and Areas for Enhancement

The primary limitation is Incorrect Mask AP of 70.5% on the Kaggle benchmark and approximately 58-65% on the external dataset. Both numbers reflect the same root cause: with only 115 training instances, the model has seen a very limited range of incorrect-wearing configurations and struggles to generalise to unseen ones. Several directions for improvement are worth pursuing: (1) synthetic data generation using diffusion models could produce diverse Incorrect Mask examples at scale, directly addressing the data scarcity problem; (2) a temporal module such as ConvLSTM could use motion context across video frames to improve detection of borderline cases; (3) INT8 quantisation would reduce memory by 4x and increase throughput to 20+ FPS on edge hardware; (4) domain adaptation techniques could reduce the observed domain shift in new deployment environments; and (5) the ephemeral ngrok URL should be replaced with persistent cloud hosting for any production use.

VIII. CONCLUSION

This paper presented MaskGuard, a real-time three-class face mask detection system for crowded environments. The system extends YOLOv11n with three custom attention modules: DS-C2f, which adds Squeeze-and-Excitation channel attention to the backbone; DIConv, which enriches the neck with four-branch parallel convolutions for multi-scale spatial context; and DAM, which applies combined channel and spatial attention in the detection head before predictions are made. An imbalance-aware training protocol combining inverse-frequency class weights, mosaic and copy-paste augmentation, minority-class oversampling, and 150 epochs of training raises Incorrect Mask AP@0.5 from 47.3% to 70.5%, a gain of 23.2 percentage points. Overall mAP@0.5 is 85.6% on validation and 84.1% on the held-out test set. Five-fold CV (mean=0.812, sigma=0.031) confirms stable generalisation across data splits. External evaluation shows above 75% mAP@0.5 on independent images, demonstrating acceptable cross-domain transfer. Live Flask/ngrok deployment processes each frame in approximately 154 ms (3.1 ms preprocess, 150.1 ms inference, 0.8 ms postprocess), corresponding to roughly 6 FPS on a T4 GPU. The full pipeline is packaged as a publicly shared, reproducible Google Colab notebook with SEED=42 fixed throughout. The most impactful next step is collecting or synthesising more varied examples of the Incorrect Mask class to improve the safety-critical minority detection.

ACKNOWLEDGEMENT

The author thanks the module supervisor for guidance and acknowledges the Kaggle community for the publicly available Face Mask Detection dataset.

REFERENCES

[1] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137–1149, Jun. 2017.

[2] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in Proc. IEEE ICCV, Venice, 2017, pp. 2980–2988.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, Las Vegas, 2016, pp. 779–788.

[4] G. Jocher et al., "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] G. Jocher et al., "YOLOv11: State-of-the-Art Object Detection," Ultralytics, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] Z. Wang et al., "FMD-YOLOv8: A face mask detection algorithm based on improved YOLOv8," Sensors, vol. 23, no. 18, p. 7753, Sep. 2023.

[7] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in Proc. IEEE CVPR, Salt Lake City, 2018, pp. 7132–7141.

[8] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in Proc. ECCV, Munich, 2018, pp. 3–19.

[9] A. Larxel, "Face Mask Detection," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/andrewmvd/face-mask-detection

[10] M. S. Ejaz et al., "Detection of face mask using YOLOv5 and YOLOv8," in Proc. IEEE ICASERT, Dhaka, 2023, pp. 1–6.

[11] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv preprint arXiv:1804.02767, 2018.

[12] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," arXiv preprint arXiv:2004.10934, 2020.

[13] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Vancouver, BC, Canada, 2023, pp. 7464–7475.

[14] C.-Y. Wang, I. H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," arXiv preprint arXiv:2402.13616, 2024.

[15] W. Liu et al., "SSD: Single shot multibox detector," in Proc. Eur. Conf. Comput. Vis. (ECCV), Amsterdam, Netherlands, 2016, pp. 21–37.

[16] M. Loey, M. Naman, and H. Zayed, "Fighting against COVID-19: A novel deep learning model based on YOLO-v2 with ResNet-50 for medical face mask detection," Sustainable Cities Soc., vol. 65, p. 102600, Feb. 2021.

[17] P. Nagrath et al., "SSDMNV2: A real-time DNN-based face mask detection system using single shot multibox detector and MobileNetV2," Sustainable Cities Soc., vol. 66, p. 102692, Mar. 2021.

[18] R. Militante and N. Dionisio, "Real-time facemask recognition with alarm system using deep learning," in Proc. IEEE 11th Int. Conf. Humanoid, Nanotechnology, Inf. Technol., Commun. Control, Environ. Manage. (HNICEM), Laoag, Philippines, 2019, pp. 1–5.

[19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), Honolulu, HI, USA, 2017, pp. 2117–2125.

[21] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW), 2020, pp. 702–703.

[22] S. Yun et al., "CutMix: Training strategy that makes mixed samples strongly regularized," in Proc. IEEE

Int. Conf. Comput. Vis. (ICCV), Seoul, South Korea, 2019, pp. 6023–6032.

[23] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), New Orleans, LA, USA, 2019.

[24] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in Proc. Int. Conf. Learn. Represent. (ICLR), Toulon, France, 2017.

[25] World Health Organization, "Advice on the use of masks in the context of COVID-19: Interim guidance," WHO, Geneva, Switzerland, Tech. Rep., Jun. 2020. [Online]. Available: https://www.who.int/publications/i/item/advice-on-the-use-of-masks-in-the-community-during-home-care-and-in-healthcare-settings-in-the-context-of-the-novel-coronavirus-(2019-ncov)-outbreak